

Implementation Plan: Profile Sync & Chat Sending Issues

Based on the analysis, here is a structured plan to resolve the state mismatch problems where the app appears online but backend operations fail.

Phase 1: Fix Profile Offline Queue Logic (Highest Priority)

Problem: The offline queue pulls the profile instead of pushing the pending local updates.

1. **Update ProfileViewModel** (/home/runner/work/Q-base/Q-base/app/src/main/java/com/algorithmx/q_base/ui/settings/ProfileViewModel.kt lines 130-137)
 - **Action:** Modify the payload created for the UPDATE_PROFILE queue item.
 - **Details:** Instead of only storing the userId, serialize the actual pending profile data (e.g., displayName, intro, avatarData) into the queue payload.
2. **Update UniversalQueueManager** (/home/runner/work/Q-base/Q-base/app/src/main/java/com/algorithmx/q_base/data/sync/UniversalQueueManager.kt lines 112-115)
 - **Action:** Change the replay logic for UPDATE_PROFILE.
 - **Details:** Parse the payload of pending changes and execute a push operation (such as ProfileRepository.updateProfile(payload)) rather than running syncUserProfile(userId), which simply overwrites local changes with stale server data.

Phase 2: Decouple Network Connectivity from Backend Session Readiness

Problem: The app behaves as if it is fully online based solely on Android network capabilities, masking underlying Appwrite session failures.

1. **Update Auth Repositories (AuthRepository.kt & AppwriteAuthImpl.kt)**
 - **Action:** Create a clear distinction between a “Cached Offline Session” and an “Active Backend Session”.
 - **Details:** When checkCurrentSession() falls back to a cached user due to a non-401 exception (like a timeout or Appwrite 500 error), it must expose a state indicating the backend is *not ready*, even though the user is locally authenticated.
2. **Update Network State Management (NetworkMonitor.kt)**

- **Action:** Combine the Android `NET_CAPABILITY_INTERNET` with an Appwrite health check.
- **Details:** The app's "Online" indicator and background sync triggers should only fire when both the network is available *and* the backend session is validated.

Phase 3: Stabilize the Sync Lifecycle

Problem: Frequent emissions from network/auth flows cancel and restart the `syncJob`, which interrupts long-running realtime connections.

1. **Optimize Flow Emissions (`MainActivity.kt` lines 121-143)**
 - **Action:** Apply `.distinctUntilChanged()` to the combined flow of network and auth states.
 - **Details:** Ensure that `syncJob` is only restarted when the state makes a meaningful transition (e.g., strictly from `Offline` to `Online`), avoiding aggressive restarts.
2. **Separate Realtime & REST Sync**
 - **Action:** Split the `syncJob` into two distinct lifecycles.
 - **Details:** The REST API flush (queue replay) can execute immediately on reconnect, but long-running realtime subscriptions (like chat listeners) should be managed in a separate `Coroutine` job that is immune to minor upstream state fluctuations.

Phase 4: Handle Missing Encryption Keys for Chat Sending

Problem: Messages fail to send silently when recipient public keys are missing, despite the app showing "online".

1. **Add Pre-flight Send Checks (`MessageSyncOutgoingExtensions.kt` lines 83-101)**
 - **Action:** Before sending, verify the presence of wrapped session keys for all participants.
 - **Details:** If keys are missing, intercept the send operation and trigger a key-fetch request for the missing recipients.
2. **Improve UI Feedback**
 - **Action:** Handle key-fetch failures gracefully.
 - **Details:** If keys cannot be fetched immediately (e.g., due to a backend failure), throw a specific exception (e.g., `MissingEncryptionKeysException`) so the `ViewModel` can display a clear message like *"Waiting for recipient encryption keys..."* instead of silently dropping the message.